Advanced Control Systems

Assignment 2: Kinetic and Potential Energy Computation, CoM Positions, Partial Jacobians, and Inertia Matrix of a PRR Manipulator

Student Name: Filippo Segalla | **Student ID:** VR538212

Department of Engineering for Innovative Medicine (DIMI)

October 20, 2026

Abstract

This report presents the dynamic modeling and energy characterization of a 3-DOF PRR (Prismatic-Revolute-Revolute) robotic manipulator defined by the URDF specification. The study details the physical rationale and analytical derivation behind every dynamic modeling phase: link Center of Mass (CoM) position vectors, partial translational and rotational Jacobians ($\mathbf{J}_P^{l_i}, \mathbf{J}_O^{l_i}$), link local inertia tensors, the configuration-dependent generalized inertia matrix $\mathbf{B}(q)$, total kinetic energy $\mathcal{T}(q, \dot{q})$, and gravitational potential energy $\mathcal{U}(q)$. Every step is derived symbolically in parametric form and evaluated numerically using MATLAB.

---

1 System Architecture & Dynamic Parameters

1.1 Description of the PRR Manipulator Dynamic Model

What is being done: We formulate the dynamic energy expressions for a spatial 3-degree-of-freedom PRR manipulator. The physical properties of the links are extracted directly from the URDF file (`00001_PRR_zzx FILIPPO SEGALLA.urdf`):

1. **Link 1 (Prismatic Joint, q_1):** A rectangular box (0.036 m $\times$ 0.036 m $\times$ 0.5 m) with mass $m_1 = 1.0$ kg.
2. **Link 2 (Revolute Joint, q_2):** A solid cylinder ($r = 0.019$ m, $h = 0.4$ m) aligned along Z_2 with mass $m_2 = 1.0$ kg.
3. **Link 3 (Revolute Joint, q_3):** A solid cylinder ($r = 0.019$ m, $h = 0.4$ m) aligned along X_3 with mass $m_3 = 1.0$ kg.

Why it is done: A complete dynamic description requires combining geometric kinematics with inertial properties (masses and inertia tensors). Establishing exact mass values and link geometry formulas is necessary to set up the Lagrangian $\mathcal{L} = \mathcal{T} - \mathcal{U}$, which governs the physical equations of motion.

1.2 Local Inertia Tensors ($\mathbf{I}_{l_i}^i$)

What is being done: For each link, the central inertia tensor $\mathbf{I}_{l_i}^i$ is evaluated with respect to a frame centered at its Center of Mass (CoM) and aligned with the link's principal axes of inertia:

- **Link 1 (Parallelepiped Box):**

$$I_{1xx} = I_{1yy} = \frac{1}{12} m_1 (0.036^2 + 0.5^2), \quad I_{1zz} = \frac{1}{12} m_1 (0.036^2 + 0.036^2) \tag{1}$$

- **Link 2 (Cylinder along Z_2):**

$$I_{2xx} = I_{2yy} = \frac{1}{12} m_2 (3 \cdot 0.019^2 + 0.4^2), \quad I_{2zz} = \frac{1}{2} m_2 (0.019^2) \tag{2}$$

- **Link 3 (Cylinder along X_3):**

$$I_{3xx} = \frac{1}{2} m_3 (0.019^2), \quad I_{3yy} = I_{3zz} = \frac{1}{12} m_3 (3 \cdot 0.019^2 + 0.4^2) \tag{3}$$

Why it is done: Inertia tensors quantify a body's resistance to rotational acceleration. By expressing inertia tensors in local central frames aligned with the principal axes, off-diagonal cross-products vanish ($\mathbf{I}_{l_i}^i = \text{diag}(I_{ixx}, I_{iyy}, I_{izz})$). When the robot moves, these constant local tensors are transformed into the global frame using rotation matrices: $\mathbf{I}_{l_i}(q) = \mathbf{R}_i(q) \mathbf{I}_{l_i}^i \mathbf{R}_i^T(q)$.

2 Centers of Mass & Partial Jacobians

2.1 Centers of Mass (CoM) Positions in Base Frame

What is being done: We locate the position vector $\mathbf{p}_{l_i}$ of each link's CoM relative to the global base frame Σ_0 using cumulative homogeneous transformation matrices:

$$\begin{bmatrix} \mathbf{p}_{l_i} \\ 1 \end{bmatrix} = \mathbf{T}_i^0(q) \begin{bmatrix} \mathbf{p}_{l_i}^i \\ 1 \end{bmatrix} \tag{4}$$

Where the local CoM vectors relative to the D-H origins are:

$$\mathbf{p}_{l_1}^1 = \begin{bmatrix} 0 \\ 0 \\ -0.25 \\ 1 \end{bmatrix}, \quad \mathbf{p}_{l_2}^2 = \begin{bmatrix} 0 \\ 0 \\ -0.20 \\ 1 \end{bmatrix}, \quad \mathbf{p}_{l_3}^3 = \begin{bmatrix} -0.20 \\ 0 \\ 0 \\ 1 \end{bmatrix} \tag{5}$$

Evaluating the transformations symbolically yields:

$$\mathbf{p}_{l_1}(q) = \begin{bmatrix} 0 \\ 0 \\ q_1 + 0.40 \end{bmatrix}, \quad \mathbf{p}_{l_2}(q) = \begin{bmatrix} 0 \\ 0 \\ q_1 + 0.85 \end{bmatrix}, \quad \mathbf{p}_{l_3}(q) = \begin{bmatrix} -0.20 \sin q_2 \cos q_3 \\ 0.20 \cos q_2 \cos q_3 \\ q_1 + 0.20 \sin q_3 + 1.05 \end{bmatrix} \tag{6}$$

Why it is done: König's theorem states that the total kinetic energy of a rigid link is the sum of translational energy at its CoM and rotational energy about its CoM. Knowing $\mathbf{p}_{l_i}(q)$ is necessary for calculating both the linear velocities of the link centers of mass and the gravitational potential energy $\mathcal{U}_i = -m_i \mathbf{g}_0^T \mathbf{p}_{l_i}$.

2.2 Partial Jacobians ($\mathbf{J}_P^{l_i}, \mathbf{J}_O^{l_i}$)

What is being done: Partial Jacobians map generalized joint velocities $\dot{q}$ to the linear velocity $\dot{\mathbf{p}}_{l_i}$ and angular velocity $\boldsymbol{\omega}_i$ of link i:

$$\dot{\mathbf{p}}_{l_i} = \mathbf{J}_P^{l_i}(q)\dot{q}, \quad \boldsymbol{\omega}_i = \mathbf{J}_O^{l_i}(q)\dot{q} \tag{7}$$

The columns j ($j \leq i$) of the partial Jacobians are computed as:

- **Prismatic Joint j:** $\mathbf{J}_{Pj}^{l_i} = \mathbf{z}_{j-1}$, $\mathbf{J}_{Oj}^{l_i} = \mathbf{0}$.
- **Revolute Joint j:** $\mathbf{J}_{Pj}^{l_i} = \mathbf{z}_{j-1} \times (\mathbf{p}_{l_i} - \mathbf{p}_{j-1})$, $\mathbf{J}_{Oj}^{l_i} = \mathbf{z}_{j-1}$.
- **Inactive Joints ($j > i$):** $\mathbf{J}_{Pj}^{l_i} = \mathbf{0}$, $\mathbf{J}_{Oj}^{l_i} = \mathbf{0}$.

Why it is done: Since link i is affected only by joints $1, \ldots, i$, columns beyond i are zero. Partial Jacobians allow us to write the kinetic energy of each link in a compact quadratic form without explicitly dealing with velocity vector products.

3 Generalized Inertia Matrix $\mathrm{B}(q)$

3.1 Mathematical Formulation

What is being done: The generalized inertia matrix $\mathbf{B}(q) \in \mathbb{R}^{3\times3}$ is constructed by summing the translational and rotational kinetic energy contributions of all links:

$$\mathbf{B}(q) = \sum_{i=1}^{3} \left(m_i (\mathbf{J}_P^{l_i})^T \mathbf{J}_P^{l_i} + (\mathbf{J}_O^{l_i})^T \mathbf{R}_i \mathbf{I}_{l_i}^i \mathbf{R}_i^T \mathbf{J}_O^{l_i} \right) \tag{8}$$

Evaluating this sum symbolically yields the parametric expression:

$$\mathbf{B}(q) = \begin{bmatrix} m_1 + m_2 + m_3 & 0 & \frac{m_3}{5}\cos q_3 \\ 0 & I_{2yy} + I_{3yy}\cos^2 q_3 + I_{3xx}\sin^2 q_3 + \frac{m_3}{25}\cos^2 q_3 & 0 \\ \frac{m_3}{5}\cos q_3 & 0 & I_{3zz} + \frac{m_3}{25} \end{bmatrix} \tag{9}$$

Substituting numerical parameters gives:

$$\mathbf{B}(q) = \begin{bmatrix} 3.0 & 0 & 0.2\cos q_3 \\ 0 & 0.1068 - 0.05324\sin^2 q_3 & 0 \\ 0.2\cos q_3 & 0 & 0.05342 \end{bmatrix} \tag{10}$$

Why it is done & Physical Properties:

1. **Symmetry ($\mathbf{B} = \mathbf{B}^T$):** Reflects the reciprocal dynamic coupling between joints.
2. **Positive Definiteness ($\mathbf{B} > 0$):** Ensures kinetic energy is strictly positive for non-zero velocities ($\mathcal{T} > 0$ for $\dot{q} \neq 0$).
3. **Coupling Term ($B_{13} = 0.2\cos q_3$):** Quantifies the inertial interaction between vertical base translation (q_1) and wrist rotation (q_3).
4. **Invariance to q_2:** $\mathbf{B}(q)$ is independent of q_2 because rotation about the vertical Z-axis does not alter the manipulator's mass distribution relative to gravity or rotation axes.

4 Kinetic & Potential Energy Computation

4.1 Total Kinetic Energy $\mathcal{T}(q, \dot{q})$

What is being done: The total kinetic energy is formulated as a quadratic expression in joint velocities:

$$\mathcal{T}(q, \dot{q}) = \frac{1}{2} \dot{q}^T \mathbf{B}(q) \dot{q} \tag{11}$$

Evaluating the matrix product numerically yields:

$$\mathcal{T}(q, \dot{q}) = 1.5\dot{q}_1^2 + 0.2\dot{q}_1\dot{q}_3\cos q_3 - 0.02662\dot{q}_2^2\sin^2 q_3 + 0.05342\dot{q}_2^2 + 0.02671\dot{q}_3^2 \tag{12}$$

Why it is done: Kinetic energy stores the mechanical work done by joint actuators during motion. The quadratic term $\frac{1}{2}B_{ii}\dot{q}_i^2$ represents the effective inertia at joint i, while cross-terms $B_{ij}\dot{q}_i\dot{q}_j$ represent dynamic cross-coupling.

4.2 Total Potential Energy $\mathcal{U}(q)$

What is being done: Potential energy due to gravity ($\mathbf{g}_0 = [0, 0, -g]^T$) is calculated as:

$$\mathcal{U}(q) = -\sum_{i=1}^{3} m_i \mathbf{g}_0^T \mathbf{p}_{l_i}(q) \tag{13}$$

Evaluating symbolically gives:

$$\mathcal{U}(q) = gm_1\left(q_1 + \frac{2}{5}\right) + gm_2\left(q_1 + \frac{17}{20}\right) + gm_3\left(q_1 + \frac{\sin q_3}{5} + \frac{21}{20}\right) \tag{14}$$

Substituting numerical values ($g = 9.81$ m/s^2, $m_i = 1$ kg):

$$\mathcal{U}(q) = 29.43q_1 + 1.962\sin q_3 + 22.56 \tag{15}$$

Why it is done: Potential energy depends strictly on position coordinates. For this PRR robot, $\mathcal{U}(q)$ depends on q_1 (vertical translation) and q_3 (elevation of Link 3), but is completely independent of q_2, since rotation about Z_0 moves the robot within a horizontal plane perpendicular to gravity.

5 MATLAB Implementation & Console Outputs

5.1 Overview of the Script (`SecondAssignment.m`)

What is being done: The script `SecondAssignment.m` implements the dynamic energy pipeline. It first computes all dynamic matrices symbolically using generic letters (m_i, I_{ixx}, g), displays the parametric expressions, and then performs numerical substitution to display clean decimal results using `vpa(..., 4)`.

Why it is done: Displaying both symbolic parametric formulas and evaluated numerical matrices allows verification of the underlying physical equations while providing concrete values for simulation and controller design.

5.2 Verified Console Logs

Below are the exact console outputs generated by executing `SecondAssignment.m` in MATLAB:

```
------------------------------------------------------------
PURELY SYMBOLIC FORMULAS (LETTERS)
------------------------------------------------------------

--- CoM Positions p_l in Base Frame Sigma_0 ---
p_l1 =
           0
           0
 q1 + 2/5

p_l2 =
            0
            0
 q1 + 17/20

p_l3 =
 -(m3*cos(q3)*sin(q2))/5
  (m3*cos(q3)*cos(q2))/5
 q1 + sin(q3)/5 + 21/20
```

```
--- Symbolic Inertia Matrix B(q) (3x3) ---
[ m1 + m2 + m3,                                                    0,
                                (m3*cos(q3))/5]
[            0, I2yy + I3yy*cos(q3)^2 + I3xx*sin(q3)^2 + (m3*cos(q3)^2)/25,
                                             0]
[ (m3*cos(q3))/5,                                                    0, I3zz + m3/25]

--- Symbolic Kinetic Energy T(q, dq) ---
(dq1*(dq1*(m1 + m2 + m3) + (dq3*m3*cos(q3))/5))/2 + (dq3*((dq1*m3*cos(q3))/5 + dq3
    *(I3zz + m3/25)))/2 + (dq2^2*(I2yy + I3yy*cos(q3)^2 + I3xx*sin(q3)^2 + (m3*cos
    (q3)^2)/25))/2

--- Symbolic Potential Energy U(q) ---
g*m1*(q1 + 2/5) + g*m2*(q1 + 17/20) + g*m3*(q1 + sin(q3)/5 + 21/20)
```

Listing 1: MATLAB Console Output: Part 1 - Purely Symbolic Formulas (Letters).

```
------------------------------------------------------------
EVALUATED NUMERICAL VALUES (DECIMALS)
------------------------------------------------------------

--- Evaluated Inertia Matrix B(q) (3x3) ---
[        3.0,                                 0,    0.2*cos(q3)]
[          0,  0.1068 - 0.05324*sin(q3)^2,              0]
[ 0.2*cos(q3),                                  0,       0.05342]

--- Evaluated Kinetic Energy T(q, dq) ---
1.5*dq1^2 + 0.2*dq1*dq3*cos(q3) - 0.02662*dq2^2*sin(q3)^2 + 0.05342*dq2^2 +
    0.02671*dq3^2

--- Evaluated Potential Energy U(q) (with g = 9.81) ---
29.43*q1 + 1.962*sin(q3) + 22.56
============================================================
```

Listing 2: MATLAB Console Output: Part 2 - Evaluated Numerical Values (Decimals).